

MATH3311, Lab Test, Term 2, 2023

Question 1.

For Python users: Copy the variables **a**, **b**, **c**, **Acheck**, **condA**, **EigA**, **PM**, **comment1**, **comment2** into Matlab Grader together with your Python code (comment out the Python code using %).

Let n be a positive integer and define the $n \times n$ matrix

$$A = [a_{i,j}]_{1 \leq i,j \leq n} = \begin{pmatrix} 0 & \frac{2}{3} & \frac{3}{4} & \frac{4}{5} & \cdots & \frac{n}{n+1} \\ \frac{2}{3} & 1 & \frac{4}{5} & \frac{5}{6} & \cdots & \frac{n+1}{n+2} \\ \frac{3}{4} & \frac{4}{5} & 2 & \frac{6}{7} & \cdots & \frac{n+2}{n+3} \\ \vdots & \vdots & \ddots & \ddots & & \vdots \\ \vdots & \vdots & & \ddots & \ddots & \frac{2n-2}{2n-1} \\ \frac{n}{n+1} & \frac{n+1}{n+2} & \frac{n+2}{n+3} & \cdots & \frac{2n-2}{2n-1} & n-1 \end{pmatrix}$$

Fix $n=10$.

- Generate A efficiently. Compute $a_{4,8}$, $a_{5,5}$ and $a_{7,3}$, store them in the variables **a**, **b** and **c**, respectively.
- Compute the 2-norm, $\|A^T A - I\|_2$. Store the value of the norm in the variable **Acheck**.
- Is A orthogonal? Write a comment in the variable 'comment1' to answer this question, i.e., type **comment1** = 'yes' if A is orthogonal or **comment1** = 'no' if not.
- Calculate the condition number of A in 2-norm using the Matlab/Python command 'cond' and store them in the variable **condA**.
- Compute the eigenvalues of A and store them in the row vector **EigA** (sorted from smallest to largest).
- Is A positive definite? Write a comment in the variable 'comment2' to answer this question, i.e., type **comment2** = 'yes' if A is positive definite or **comment2** = 'no' if not.
- Choose $n=4$ in this part. Find a permutation matrix P such that $PA=LU$, where L is a unitary lower triangular matrix and U is an upper triangular matrix. For Python users, store P in the matrix **PM** = [...;...;...;...], that is, the rows of **PM** are separated by ; , and

the entries in each row are separated by a space.

<u>Matlab Code</u>	<u>Python Code</u>
<pre> n = 10; for i =1:n for j=1:n A(i,j)=(i+j-1)/(i+j); end A(i,i)=i-1; end a=A(4,8); b=A(5,5); c=A(7,3); Acheck = norm(A'*A-eye(n)); condA = cond(A); EigA=eig(A)'; [L,U,PM] = lu(A(1:4,1:4)); </pre>	<pre> from numpy.linalg import cond, eigvals from numpy import zeros, identity, sort from scipy.linalg import norm, lu n= 10 A=zeros((n,n)) for i in range(0,n): for j in range(0,n): A[i,j]=(i+j+1)/(i+j+2) A[i,i]=i a=A[3,7] b=A[4,4] c=A[6,2] Acheck = norm(A.T@A-identity(n),2) condA = cond(A) EigA=eigvals(A) EigA=sort(EigA) PM, L, U = lu(A[0:4,0:4]) PM = PM.T </pre>

```

comment1 = 'no';
comment2 = 'no';

```

Question 2.

For Python users: Copy the variables **x0, x1, x2, x3, xe, invf2** into Matlab Grader together with your Python code (comment out the Python code using %).

The Newton method can be used to find a root of a given function g , i.e. a value x^* such that $g(x^*) = 0$. It is defined by the iteration

$$x_{n+1} = x_n - \frac{g(x_n)}{g'(x_n)}, \quad n = 0, 1, 2, 3, \dots$$

Let $f(x) = (x - 1)^4 + \exp(x)$, for $x \in [0, 3]$.

The aim is to find $f^{-1}(2)$, using Newton's method.

- Define a variable **x0** and set it to 3. Use this value as starting value for the Newton iteration.
- Modify the above Newton's iteration so that it can be used to approximate the inverse of a function. Write a Matlab/Python program which uses this modified Newton iteration (do Not use build-in functions like `fzero` in Matlab or `fsolve` in Python to do that).

Compute x_1 , x_2 , and x_3 , and store the results in **x1**, **x2**, and **x3**, respectively.

c) Run the Newton iteration till the difference between successive approximations is smaller than the machine epsilon eps, i.e. $|x_{n+1} - x_n| < 2^{-52}$. Store the result in **xe**.

d) Find $f^{-1}(2)$. Store the result in **invf2**.

<u>Matlab Code</u>	<u>Python Code</u>
<pre> g = @(x)(x-1).^4 +exp(x)-2; dg= @(x) 4*(x-1).^3 + exp(x); xapp=3; xx(1)=xapp; for n=2:4; xx(n)=xx(n-1)-g(xx(n-1))/dg(xx(n-1)); end xappb=xx(2); while abs(xapp-xappb) > 10^(-15) xappn= xappb - g(xappb)/dg(xappb); xapp = xappb; xappb = xappn; end x0=xx(1); x1=xx(2); x2=xx(3); x3=xx(4); xe=xappn; invf2=xe; </pre>	<pre> from numpy import zeros, exp def g(x): return (x-1)**4 +exp(x)-2 def dg(x): return 4*(x-1)**3 +exp(x) xapp=3 xx=zeros(4) xx[0]=xapp for n in range(1,4): xx[n]=xx[n-1]-g(xx[n-1])/dg(xx[n-1]) xappb=xx[1] while abs(xapp-xappb) > 10**(-15): xappn= xappb - g(xappb)/dg(xappb) xapp = xappb xappb = xappn x0=xx[0] x1=xx[1] x2=xx[2] x3=xx[3] xe=xappn invf2=xe </pre>

Question 3.

For Python users: Copy the variables **N**, **x**, **w**, **f**, **Qf**, **Qfs (from Part A)**, **errtrap**, **errsimp** into Matlab Grader together with your Python code (comment out the Python code using %).

In this question, the goal is to use the composite trapezoidal and the composite Simpson's rules (defined below) to approximate the integral

$$I := \int_0^1 3^x (\sin(\pi x))^2 dx.$$

Recall that the composite trapezoidal rule for an integrand g is given by

$$\int_a^b g(x)dx \approx h[g(x_0)/2 + g(x_1) + g(x_2) + \cdots + g(x_{N-2}) + g(x_{N-1}) + g(x_N)/2],$$

Recall that the composite Simpson's 1/3 rule for an integrand g is given by: for N even,

$$\int_a^b g(x)dx \approx \frac{h}{3} [g(x_0) + g(x_N) + 4g(x_{N-1}) + 2 \sum_{i=1}^{N/2-1} [2g(x_{2i-1}) + g(x_{2i})]],$$

In the above rules, $h=(b-a)/N$ and the equally spaced points $x_n=a+nh$ for $n=0,1,2,\dots,N$

Part A

- 1) Set $N=6$ and store it in the variable **N** and define a variable **h** given by $h=(b-a)/N$.
- 2) Compute an approximation to the integral I using **f** and the composite trapezoidal rule using $N+1$ points in the following way:
 - 2.1) Define a row vector **x** of $N+1$ equally spaced points whose first component is 0 and whose last component is 1
 - 2.2) Define a row vector **w** given by $[1, 2, 2, \dots, 2, 2, 1]$ of length $N+1$.
 - 2.3) Compute the function $f(x) = 3^x (\sin(\pi x))^2$ at the points $x_0, x_1, \dots, x_N$, and store your results in the row vector variable **f**.
 - 2.4) Store the value of this approximation in the variable **Qf**.
- 3) Compute an approximation to the integral I using **f** and the composite Simpson's 1/3 rule using N intervals (i.e. $N+1$ points). Store the value of this approximation in the variable **Qfs**.

Part B

Be careful not to change the values of N, x, w, f, Qf, Qfs from Part A.

- 4) Use the true solution $\frac{4\pi^2}{(4\pi^2 + (\ln 3)^2) \ln 3}$ to compute the absolute errors from both trapezoidal and Simpson's rules, when $N=30$. Store the absolute error for the trapezoidal rule in **errtrap** and for the Simpson rule in **errsimp**.

<u>Matlab Code</u>	<u>Python Code</u>
<pre> N = 6; h = 1/N; x = linspace(0,1,N+1); f = 3.^x.*(sin(pi*x)).^2; w = [1 2*ones(1,N-1) 1]; Qf = h/2*w*f'; ws = [1 2*ones(1,N-1) 1]; ws(2:2:N) = 4; Qfs = h/3*ws*f'; N1 = 30; h = 1/N1; x1 = linspace(0,1,N1+1); f1 = 3.^x1.*(sin(pi*x1)).^2; w1 = [1 2*ones(1,N1-1) 1]; Qf1 = h/2*w1*f1'; w1(2:2:N1) = 4; Qfs1 = h/3*w1*f1'; trsol = (4*pi^2)/(log(3)*(4*pi^2 + log(3)^2)); errtrap = abs(Qf1 - trsol); errsimp = abs(Qfs1- trsol); </pre>	<pre> from numpy import log, ones, pi, sin, linspace N = 6 h = 1/N x = linspace(0,1,N+1) f = 3**x*(sin(pi*x))**2 w = 2*ones(N+1) w[0]=1 w[N]=1 Qf = h/2*w@(f.T) ws = 2*ones(N+1) ws[0]=1 ws[N]=1 ws[1::2] = 4 Qfs = h/3*ws@(f.T) N1 = 30 h1 = 1/N1 x1 = linspace(0,1,N1+1) f1 = 3**x1*(sin(pi*x1))**2 w1 = 2*ones(N1+1) w1[0]=1 w1[N1]=1 Qf1 = h1/2*w1@(f1.T) w1[1::2] = 4 Qfs1 = h1/3*w1@(f1.T) trsol = (4*pi**2)/(log(3)*(4*pi**2+log(3)**2)); errtrap = abs(Qf1 - trsol) errsimp = abs(Qfs1- trsol) </pre>

Question 4.

For Python users: Copy the variables **tdat**, **ydat**, **teval**, **yspteval**, **rw**, **a0**, **a**, **b**, **yteval** into Matlab Grader together with your Python code (comment out the Python code using %). The goal is to estimate the gold price data using a cubic spline & a linear least square method.

Time (years)	Gold price (dollars)
1	106.236
2	183.683
3	139.279
4	160.48
5	207.895

6	463.666
7	388.06
8	407.674
9	442.974
10	509.423

- a) Store the first column in the vector **tdat** and the second column in the vector **ydat**.
- b) Compute the not-a-knot cubic spline which interpolates the data. Evaluate the spline at **teval**=[1.5,2.5,3.5,...,9.5] and store the results in the row vector **yspteval**.
- c) Let w (with entries w_i for $i=1,2,3,\dots,10$) be the complex DFT of the data **ydat**, it can be calculated using the Matlab\Python function `fft`. Find the real part of w and store the result in the row vector **rw**. Hint: For Matlab users, you need to use the command 'real(x)' to find the real part of x . For Python users, you need to use the command 'x.real' to find the real part of x .
- d) The number of data points $m=10$. Let $p=m/2=5$. Calculate $a_0 = w_1/m$ and store the result in **a0**

- e) Calculate the row vector **a** with entries $a_k = \frac{2}{m} \Re w_{k+1}$ for $k=1,2,3,4$ and $a_5 = \frac{w_6}{m}$, where $\Re w_{k+1}$ is the real part of w_{k+1} .

- f) Calculate the row vector **b** with entries $b_k = -\frac{2}{m} \Im w_{k+1}$ for $k=1,2,3,4$, and $b_5 = 0$, and where $\Im w_{k+1}$ is the imaginary part of w_{k+1} . Hint: For Matlab users, you need to use the command 'imag(x)' to find the imaginary part of x . For Python users, you need to use the command 'x.imag' to find the imaginary part of x .

- g) Calculate the DFT approximation (below) and store the results in the row vector **yteval**.

$$a_0 + \sum_{k=1}^p a_k \cos(2\pi k \text{ teval} / m) + b_k \sin(2\pi k \text{ teval} / m)$$

Matlab Code

```
tdat=[1 2 3 4 5 6 7 8 9 10];
ydat = [106.2360 183.6830 139.2790 160.4800 207.8950 ...
463.6660 388.0600 407.6740 442.9740 509.4230];
teval=1.5:1:9.5;
%Cubic spline
pp = spline(tdat,ydat);
yspteval=ppval(pp,teval);
%DFT
```

```

m=10; p=5;
w=fft(ydat);
rw=real(w);
a(1:4)=2*real(w(2:5))/m; b(1:4)=-2*imag(w(2:5))/m;
a0=w(1)/m; a(5)=w(6)/m; b(5)=0;
yteval=a0;
for i=1:p;
yteval=yteval+a(i)*cos(2*pi*i/m*teval)+b(i)*sin(2*pi*i/m*teval);
end

```

Python Code

```

from numpy import linspace, pi, array, sin, cos
from scipy.interpolate import CubicSpline
from scipy.fft import fft
tdat=array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
ydat = array([106.2360, 183.6830, 139.2790, 160.4800, 207.8950, 463.6660,
388.0600, 407.6740, 442.9740, 509.4230])
teval = linspace(1.5,9.5,9)
#Cubic Spline
pp = CubicSpline(tdat, ydat)
yspteval = pp(teval)
# DFT
m=10; p=5
w=fft(ydat)
rw=w.real
a=2/m*rw[1:6]; b =-2/m*w.imag[1:6]; a0=w[0]/m
a[4]=w[5]/m; b[4]=0
yteval=a0
for i in range(1,p+1):
    yteval=yteval+a[i-1]*cos(2*pi*i/m*teval)+b[i-1]*sin(*pi*i/m*teval)

```

Question 5.

For Python users: Copy the variables **h** (when $N = 4$), **x** (from part b when $N = 4$), **U4** (from part e when $N = 4$), **A** and **a** (from part d when $N = 4$), **E**, **OC** into Matlab Grader together with your Python code (comment out the Python code using %).

In this question we use a finite difference scheme to approximate the solution to the BVP: $u''(x)-u(x)=f(x)$ for $0 < x < 1$, with $u(0)=0$ and $u(1)=I$, where $f(x) = -(\pi^2 + 1) \sin(\pi x) - x$.

Part A

a) Set $N = 4$;

b) Define the points $x_n = nh$, for $n = 0, 1, \dots, N$, where $h = 1/N$ and save them in the

vector $\mathbf{x}$.

c) Approximate $u(x_n) \approx U_n$ for $n = 1, 2, \dots, N-1$ with $U_0 = 0$ and $U_N = 1$. (There is no Matlab/Python command to write in this part and this part is not marked.)

d) Use the finite difference scheme $U_{n+1} - 2U_n + U_{n-1} - h^2 U_n = h^2 f(x_n)$ for $1 \leq n \leq N-1$, to obtain a system of linear equations $A\mathbf{u} = \mathbf{a}$ where A is an $(N-1) \times (N-1)$ matrix and $\mathbf{a}$ is a column vector of length $N-1$, given by $\mathbf{a} = (h^2 f(x_1), h^2 f(x_2), h^2 f(x_3) - 1)^\top$. Store the matrix A in the variable $\mathbf{A}$ and the right-hand side $\mathbf{a}$ in the column vector $\mathbf{a}$.

e) Solve the system of linear equations and store the results in the row vector $\mathbf{U4}$, given by the values $[U_1, U_2, U_3]$.

Part B

Be careful not to change the values of the variables $\mathbf{h}$, $\mathbf{x}$, $\mathbf{U4}$, $\mathbf{A}$, $\mathbf{a}$ from **Part A**.

f) The true solution of the given boundary-value problem is $u(x) = \sin(\pi x) + x$. Define a row vector $\mathbf{u}$ of length $N-1$ with entries $u(x_1), u(x_2), \dots, u(x_{N-1})$. Compute the infinity norm error between the true vector solution $\mathbf{u}$ and the approximate vector solution $\mathbf{U}$ for $N = 80, 160, 320$. Store your results in the row vector $\mathbf{E} = [E_1, E_2, E_3]$ respectively.

g) Estimate the order of convergence (OC) using the formula
$$OC_i = \frac{\log\left(\frac{E_i}{E_{i+1}}\right)}{\log(2)} \quad \text{for } i = 1, 2$$
 and store your results in the row vector $OC = [OC_1 \ OC_2]$.

Matlab Code

```
N = 4;
x = linspace(0,1,N+1);
U4 = zeros(N-1,1);
h=1/N;
d3=ones(N-1,1)*(-2-h^2);
d4=ones(N-2,1);
A=diag(d4,-1)+diag(d3)+diag(d4,1);
f(1:N-2)=((pi^2+1)*sin(pi*x(2:N-1))+x(2:N-1));
a=-[h^2*f(1:N-2) h^2*((pi^2+1)*sin(pi*x(N))+x(N))+1]';
w=A\ a;
U4=w;

for i=1:3;
N=2^i*40;
y = linspace(0,1,N+1);
```



```

U = zeros(N-1,1);
h1=1/N;
d3=ones(N-1,1)*(-2-h1^2);
d4=ones(N-2,1);
A1=diag(d4,-1)+diag(d3)+diag(d4,1);
F1(1:N-2)=(pi^2+1)*sin(pi*y(2:N-1))+y(2:N-1);
a1=-[h1^2*f1(1:N-2) h1^2*((pi^2+1)*sin(pi*y(N))+y(N))+1]';
U=A1\ a1;
u= (sin(pi*y(2:N))+y(2:N))';
E(i)=norm(U-u,inf);
end

OC(1)=log(E(1)/E(2))/log(2);
OC(2)=log(E(2)/E(3))/log(2);
OC=[OC(1) OC(2)]

```

Python Code

```

from numpy.linalg import solve, norm
from numpy import diag, log, ones, zeros, pi, inf, sin
from numpy import linspace

N=4
x = linspace(0,1,N+1)
U4 = zeros(N-1)
h=1/N
d4=ones(N-2)
d3=ones(N-1)*(-2-h**2)
A=diag(d4,-1)+diag(d3)+diag(d4,1)
a=zeros(N-1)
a[:-1]=-h**2*((pi**2+1)*sin(pi*x[1:-2])+x[1:-2])
a[N-2]=-h**2*((pi**2+1)*sin(pi*x[N-1])+x[N-1])-1

U4=solve(A,a)
E=zeros(3)
OC=zeros(2)
for i in range(0,3):
    N=(2**i)*80
    x1 = linspace(0,1,N+1);
    U = zeros(N)
    h1=1/N
    d4=ones(N-2)
    d3=ones(N-1)*(-2-h1**2)
    A1=diag(d4,-1)+diag(d3)+diag(d4,1)

```

```

a1=zeros(N-1)
a1[:-1]=-h1**2*((pi**2+1)*sin(pi*x1[1:-2])+x1[1:-2])
a1[N-2]=-h1**2*((pi**2+1)*sin(pi*x1[N-1])+x1[N-1])-1
U=solve(A1,a1)
u= (sin(pi*x1[1:N])+x1[1:N]).T
E[i]=norm(U-u,inf)

OC[0]=log(E[0]/E[1])/log(2)
OC[1]=log(E[1]/E[2])/log(2)

```